

7장 중간 코드로 변환

최광훈
전남대학교

7.1 중간 표현 트리(Intermediate Representation Trees)

추상 구문을 바로 기계어로 번역하면, 컴파일러 이식성과 모듈성 문제 발생.

- 따라서, Intermediate Representation, IR을 사용
- Front end: 소스 언어 $\rightarrow$ IR, Back end: IR $\rightarrow$ 기계어

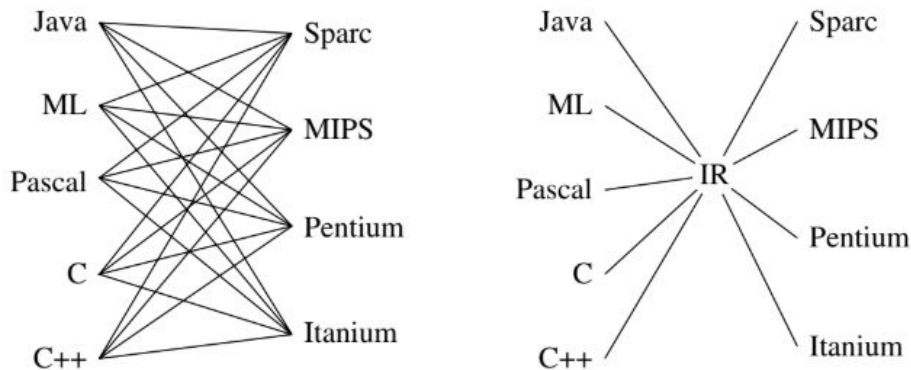


FIGURE 7.1. Compilers for five languages and four target machines:
(a) without an IR, (b) with an IR.

7.1 중간 표현 트리(Intermediate Representation Trees)

좋은 IR의 조건

- 생성하기 쉽고, 기계어로 번역하기 좋으며, 의미가 단순하고 명확해서 최적화에 적합

이 장에서는 IR로 **simple expression trees**를 사용하며, 복잡한 구문을 **fetch, store, add, move, jump** 같은 단순한 추상 기계 명령들의 조합으로 표현

```

package Tree;

abstract class Exp
  CONST(int value)
  NAME(Label label)
  TEMP(Temp.Temp temp)
  BINOP(int binop, Exp left, Exp right)
  MEM(Exp exp)
  CALL(Exp func, ExpList args)
  ESEQ(Stm stm, Exp exp)

abstract class Stm
  MOVE(Exp dst, Exp src)
  EXP(Exp exp)
  JUMP(Exp exp, Temp.LabelList targets)
  CJUMP(int relop, Exp left, Exp right, Label iftrue, Label iffalse)
  SEQ(Stm left, Stm right)
  LABEL(Label label)

other classes:
  ExpList(Exp head, ExpList tail)
  StmList(Stm head, StmList tail)

other constants:
  final static int BINOP.PLUS, BINOP.MINUS, BINOP.MUL, BINOP.DIV, BINOP.AND,
    BINOP.OR, BINOP.LSHIFT, BINOP.RSHIFT, BINOP.ARSHIFT, BINOP.XOR;

  final static int CJUMP.EQ, CJUMP.NE, CJUMP.LT, CJUMP.GT, CJUMP.LE,
    CJUMP.GE, CJUMP.ULT, CJUMP.ULE, CJUMP.UGT, CJUMP.UGE;

```

IR을 위한 Java 구현
- chap7/Tree/*.java

(주의) Exp 클래스(식) vs EXP 클래스
(문장), 윈도우즈 파일시스템에서
대소문자를 구분하지 않음

FIGURE 7.2. Intermediate representation trees.

7.1 중간 표현 트리(Intermediate Representation Trees)

abstract class Exp

- CONST(*i*)
- NAME(*n*)
- TEMP(*t*)
- BINOP(*o*, exp1, exp2)
- MEM(exp)
- CALL(*f*, expList)
- ESEQ(stm, exp)

IR trees: Expressions

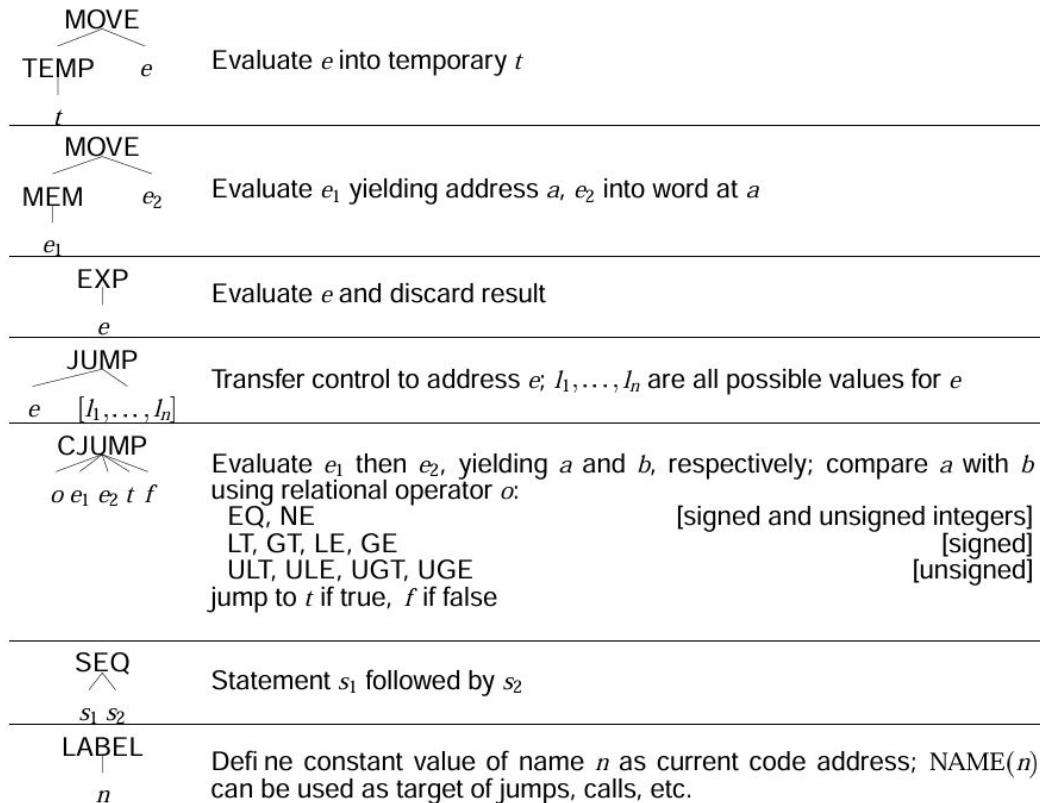
CONST <i>i</i>	Integer constant <i>i</i>
NAME <i>n</i>	Symbolic constant <i>n</i> [a code label]
TEMP <i>t</i>	Temporary <i>t</i> [one of any number of "registers"]
BINOP <i>o</i> <i>e</i> ₁ <i>e</i> ₂	Application of binary operator <i>o</i> : PLUS, MINUS, MUL, DIV AND, OR, XOR [bitwise logical] LSHIFT, RSHIFT [logical shifts] ARSHIFT [arithmetic right-shift] to integer operands <i>e</i> ₁ (evaluated first) and <i>e</i> ₂ (evaluated second)
MEM <i>e</i>	Contents of a word of memory starting at address <i>e</i>
CALL <i>f</i> [<i>e</i> ₁ , ..., <i>e</i> _{<i>n</i>}]	Procedure call; expression <i>f</i> is evaluated before arguments <i>e</i> ₁ , ..., <i>e</i> _{<i>n</i>}
ESEQ <i>s</i> <i>e</i>	Expression sequence; evaluate <i>s</i> for side-effects, then <i>e</i> for result

7.1 중간 표현 트리(Intermediate Representation Trees)

abstract class Stm

- MOVE(dstExp, srcExp)
- EXP(exp)
- JUMP(exp, labList)
- CJUMP(relop, ...)
- SEQ(leftStm, rightStm)
- LABEL(label)

IR trees: Statements



7.1 중간 표현 트리(Intermediate Representation Trees)

다른 클래스

- ExpList (headExp, tailExpList)
- StmList (headStm, tailStmList)

상수

- BINOP.PLUS, BINOP.MINUS, ...
- CJUMP.EQ, CJUMP.NE, ...

7.1 중간 표현 트리(Intermediate Representation Trees)

Tree IR 시맨틱스

- Tree IR은 거의 **formal semantics**를 줄 수 있을 만큼 단순
- 그러나 **procedure/function** 정의 전체를 표현하지는 않고, 각 **function body**만 표현
- 함수 진입/종료 코드는 **target machine**마다 다르므로, 나중에 **machine-specific glue code**로 추가함 (함수 호출시 인자 전달은 7장 Tree IR 변환에서 고려하지 않음)

7.2 Tree IR로 변환

각 메소드 변환

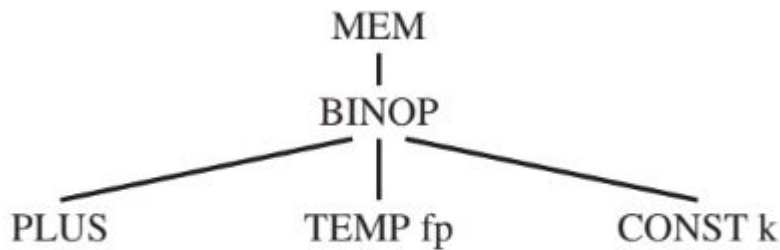
- 스택 프레임 생성
- **formal** 파라미터와 지역 변수 준비 (환경: 변수 이름 -> 스택 프레임의 **Access**)
- 문장 번역
- 리턴문 번역

7.2 Tree IR로 변환

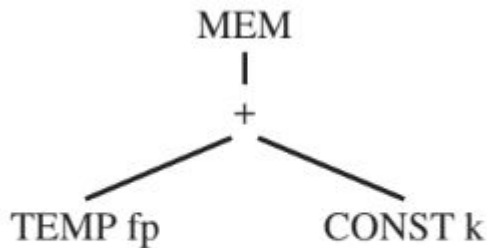
변수 v (simple variable)

- k : 변수 v 의 스택 프레임에서 오프셋
- **TEMP fp**: 프레임 포인터 레지스터

MiniJava에서 모든 변수는 워드 크기(word size)로 가정



$\text{MEM}(\text{BINOP}(\text{PLUS}, \text{TEMP fp}, \text{CONST } k))$



$+(\text{TEMP fp}, \text{CONST } k)$

7.2 Tree IR로 변환

Frame 클래스

- CPU에 속하는 정의(FP, wordSize)

```
package Frame;
public class Frame {
    :
    abstract public Temp FP();
    abstract public int wordSize();
}
public abstract class Access {
    public abstract Tree.Exp exp(Tree.Exp framePtr);
}
```

프레임 포인터 주소(framePtr)를 인자로 받아
해당 변수의 Access를 중간 표현 식 Exp로 변환

예) Access a: InFrame(k)

a.exp(new TEMP(frame.FP())) =>
MEM(
 BINOP(PLUS, TEMP(frame.FP()), CONST(k))
)

예) Access b: InReg(t832) => TEMP(t832)

7.2 Tree IR로 변환

배열 변수

- 배열은 배열 시작 주소(포인터)
- $e[i] : \text{MEM}(+(e, x(i, \text{CONST } w)))$

e : 배열

i : 첨자

w : 단어 크기

MiniJava 배열

- 배열 변수는 포인터
 - 배열 상수 `int arr[10]`을 선언할 수 없음
- ```
int [] arr;
arr = new int[12];
brr = new int[12];
arr = brr
```
- 배열 생성하면 모든 요소를 0으로 초기화
  - `arr = brr`; `arr`과 `brr`은 동일한 배열 공간을 가리키도록 주소를 복사

(선택) 배열 첨자  $i < \text{size}(e)$  조건을 런타임에 검사

cf. MiniJava 객체

- 포인터
- 객체 할당도 배열 할당과 마찬가지로 포인터 복사 (필드 복사 X)

## 7.2 Tree IR로 변환

**l-value** : 대입문의 왼쪽에 올 수 있는 식 (값이 저장될 수 있는 메모리 위치)

- 예:  $x$ ,  $p.y$ ,  $a[i+2]$

**r-value**: 대입문의 오른쪽 값으로만 사용되는 식 (계산된 값, 저장 위치는 아님)

- 예:  $a+3$ ,  $f(x)$

## 7.2 Tree IR로 변환

### MiniJava

- 변수와 l-value는 모두 'scalar'
- 정수 또는 포인터, 1개 워드 크기, 레지스터에 저장 가능
- 배열과 객체도 포인터로 취급

(참고) 'scalar'가 아닌(structured) l-value

- C의 struct, Pascal의 array/record
- 여러 워드 크기, 주소와 크기 정보 필요
- MEM( exp, size ) 또는 MEM( +(TEMP fp, CONST  $k_n$ ), size)

## 7.2 Tree IR로 변환

배열 원소 접근:  $a[i]$

- 기준 주소(base address)에서  $i$ 번째 원소의 위치를 계산
- 주소( $a[i]$ ) =  $(i - 1) * s + a$       (참고:  $a - s \times 1$ 은 컴파일 시점에 계산 가능)

레코드 필드 선택:  $a.f$

- 레코드 안에서 필드  $f$ 가 시작하는 위치
- 주소( $a.f$ ) =  $a + \text{offset}(f)$

## 7.2 Tree IR로 변환

배열 변수 **a**도 l-value

- 배열 의 기준 주소:  $+(TEMP\ fp, CONST\ k)$

MiniJava 배열 원소 **a[i]**:

- $MEM\ ( \ +(MEM(e), BINOP(MUL, i, CONST\ W) )$
- **e** : 배열 기준 주소

(\*) 런타임 배열 첨자 범위 검사

## 7.2 Tree IR로 변환

정수 수식

- $a+b$ : `BINOP(PLUS, a, b)`
- $-x$ : `0-x` `BINOP(MINUS, CONST 0, x)`
- $\sim x$ : `x XOR all_ones`

부동소수점 negation 문제

- 부동소수점 표현에는  $+0.0$ 과  $-0.0$ 이 존재
- negation은 두 값을 서로 바꾸어야 함

## 7.2 Tree IR로 변환

비교 연산  $x < 5$

- CJUMP(LT, x, CONST(5), t, f)

- t와 f: 라벨

조건식 if e1 then e2 else e3

- seq ( CJUMP(NE, e1, CONST(0), lt, lf)

    LABEL(lt), e2, JUMP(le),

    LABEL(lf), e3,

    LABEL(le) )

단순 번역은 비효율적일 수 있음

- branch가 문장인 경우
- branch가 다시 조건식인 경우

조건식 안의 조건식 : if  $x < 5$  then  $a > b$  else 0

- 불필요하게 0/1값을 만들지 않고 점프 구조로

(교재 설명 참고)

## 7.2 Tree IR로 변환

문자열 “hello world”

- NAME(lbl)
- .word 11
- lbl: .ascii “hello world”

## 7.2 Tree IR로 변환

레코드 생성 { e1, e2, ..., en }

- n개 필드의 레코드를 힙에 생성
- 각 필드는 e1, e2, ..., en의 값으로 초기화
- ESEQ( SEQ(MOVE(TEMP r, externalCall("allocRecord", [CONST n])),

SEQ(MOVE(MEM(TEMP r), e1),

SEQ(...,

MOVE(MEM(+(TEMP r, CONST (n-1)w)), en))),

TEMP r)

```
r <- alloc(n * W)
MEM(r + 0*W) <- e1
...
MEM(r + (n-1)*W) <- en
result = TEMP r
```

## 7.2 Tree IR로 변환

배열 생성  $t[e1]$  of  $e2$

- `externalCall("initArray", [e1, e2])`

## 7.2 Tree IR로 변환

while 반복문

- test:   if not (condition) goto done

    body

    goto test

done:

- body에 break문이 있으면, done으로 분기하는 문장으로 변환

## 7.2 Tree IR로 변환

for 반복문

- while 반복문의 변환

```
for (i=lo; i<=hi; i++) {
 // body
}
```

```
i=lo;
limit=hi;
while (i<=limit) {
 // body
 i++;
}
```

- 주의)  $\text{limit} = \text{maxint}$ 인 경우  $i+1$  식에서 **overflow** 발생

## 7.2 Tree IR로 변환

함수 호출  $f(a_1, \dots, a_n)$

- CALL (NAME  $l_f$ , [  $e_1, e_2, \dots, e_n$  ])

- $l_f$ 는 함수  $f$  코드의 시작 라벨

- 객체지향 언어 **this**를 첫번째 인자로 추가

$$p.m(a_1, \dots, a_n) \Rightarrow \text{CALL}(\text{NAME } l_{c\$m}, [p, e_1, \dots, e_n])$$

(클래스  $c$ 의 객체  $p$ ,  $c\$m$ 은 클래스  $c$ 의 메소드  $m$ )

MiniJava는 컴파일 시점에 결정되는 **static method**만 허용(클래스의 메서드로 특정)

14장에서 객체지향 언어의 **dynamic method** 컴파일 방법 소개 (객체의 메서드로 특정)

## 7.3 선언: 변수 정의

“ `int x = e;` ”를 번역할 때 두 가지 산출물이 필요

- 변수 선언으로 이후 변환의 타입 환경을 확장
- 초기화 식(**e**)는 실행해야할 코드로 변환하고 함수 몸체 앞에서 실행되어야 함
- 따라서, 초기화 코드를 변환의 결과로 리턴하여 나중에 함수 몸체 앞에 배치

(참고: 함수나 타입 선언은 특별한 의미가 없는 “no-op” 코드를 리턴)

## 7.3 선언: 함수 정의

MiniJava의 함수 선언을 IR로 번역한 뒤, 그 IR을 어셈블리 함수 형태 (프롤로그/바디/에필로그)로 내보내기 (Translate/Frame 모듈의 역할 분담)

- 함수 본문(body)은 표현식(expression) 이고, 그 표현식을 IR로 번역
- 하지만 실제 어셈블리 함수가 되려면 호출 규약 때문에 프롤로그/에필로그(스택 프레임 관리, 인자 이동, 레지스터 저장/복구 등)가 반드시 필요
- 그 주변부 작업을 Frame 모듈의 procEntryExit\* 함수들이 단계적으로 담당
- 최종 산출물은 함수 1개당 ProcFrag(프로시저 프래그먼트)로 모아 두고,
- 뒤 단계(코드생성/레지스터할당)가 이를 처리

## 7.3 선언: 함수 정의

어셈블리로 작성하는 함수의 일반적인 구성: 프로로그 + 몸체 + 에필로그

| 프로로그                                                                                                         |
|--------------------------------------------------------------------------------------------------------------|
| 1. 함수 시작을 알리는 <b>Pseudo</b> 명령어                                                                              |
| 2. 함수 이름 레이블 정의                                                                                              |
| 3. 스택 포인터 조정 (새 프레임 할당)                                                                                      |
| 4. 인자( <b>formal args.</b> ) 처리<br>- <b>escaping</b> 인자: 스택 프레임에 저장<br>- <b>non-escaping</b> 인자: 임시 레지스터로 이동 |
| 5. 함수 안에서 사용한 <b>callee-save</b> 레지스터 저장<br>(반드시 복구해야하는 레지스터+리턴 주소 레지스터)                                     |

| 몸체          |
|-------------|
| 6. 함수 본문 코드 |

| 에필로그                              |
|-----------------------------------|
| 7. 반환값을 지정한 레지스터( <b>RV</b> )로 옮김 |
| 8. <b>call-save</b> 레지스터 복구       |
| 9. 스택 포인터 원복(프레임 해제)              |
| 10. 리턴 (리턴 주소로 점프)                |
| 11. 함수 끝 <b>Pseudo</b> 명령어        |

## 7.3 선언: 함수 정의

중간 코드 변환 단계에서 프레임 크기를 알 수 없고, 나중에 레지스터 할당 이후 알려짐!

| <del>procEntryExit* 함수를 도입해서 처리</del><br>프로로그                                                       | 몸체          |
|-----------------------------------------------------------------------------------------------------|-------------|
| 1. 함수 시작을 알리는 <b>Pseudo</b> 명령어                                                                     | 6. 함수 본문 코드 |
| 2. 함수 이름 레이블 정의                                                                                     |             |
| 3. 스택 포인터 조정 (새 프레임 할당)                                                                             |             |
| 4. 인자(formal args.) 처리<br>- <b>escaping</b> 인자: 스택 프레임에 저장<br>- <b>non-escaping</b> 인자: 임시 레지스터로 이동 |             |
| 5. 함수 안에서 사용한 <b>callee-save</b> 레지스터 저장 (반드시 복구해야하는 레지스터+리턴 주소 레지스터)                               |             |

| 에필로그                              |
|-----------------------------------|
| 7. 반환값을 지정한 레지스터( <b>RV</b> )로 옮김 |
| 8. <b>call-save</b> 레지스터 복구       |
| 9. 스택 포인터 원복(프레임 해제)              |
| 10. 리턴 (리턴 주소로 점프)                |
| 11. 함수 끝 <b>Pseudo</b> 명령어        |

## 7.3 선언: 함수 정의

Tree.Stm `procEntryExit1`(Tree.Stm body) (7장)

- Tree IR(중간표현) 수준에서 view shift + callee-save 처리를 IR로 삽입

Assem.InstrList `procEntryExit2`(Assem.InstrList body) (9장)

- Assem(가상 어셈블리 instruction list) 수준에서 레지스터 할당기에게 “끝에서 살아있는 레지스터들”을 알려주는 sink를 추가

Proc `procEntryExit3`(Assem.InstrList body) (12장)

- `procEntryExit3`: 함수 프로로그와 에필로그로 감싸서 최종 Proc을 만들기

## 7.3 선언: 함수 정의

Tree.Stm **procEntryExit1**(Tree.Stm body) (7장)

- Tree IR(중간표현) 수준에서 **view shift + callee-save** 처리를 IR로 작성
- 들어오는 인자(formal args)를 함수 내부에서 보이는 위치로 옮김(view shift)

예) a0 -> temp\_x, a1 -> frame[+offset]

- **callee-save** 저장과 복구 코드를 IR에 추가

전략1) 모든 **callee-save** 레지스터를 함수 시작할 때 프레임에 저장하고 끝날 때 복구

전략2) 각 **callee-save** 레지스터 마다 새 **temp**를 만들고, 함수 시작할 때 그 레지스터를 이 **temp**에 저장하고 끝날 때 복구 (**Spill**을 지원하는 레지스터 할당에서 30  
필요한 경우에만 프레임으로 옮김)

## 7.3 선언: 함수 정의

`Assem.InstrList` `procEntryExit2`(`Assem.InstrList` body) (9장)

- `Assem`(가상 어셈블리 instruction list) 수준에서 레지스터 할당기에게 “끝에서 살아있는 레지스터들”을 알려주는 sink를 추가

예) `new Assem.OPER("", null, returnSink(ZERO, RA, SP, calleeSaves, ...))`

## 7.3 선언: 함수 정의

Proc `procEntryExit3`(Assem.InstrList body) (12장)

- `procEntryExit3`: 함수 프로로그와 에필로그로 감싸서 최종 Proc을 만들기

- 함수 시작 `pseudo` 명령어 (예: `.global f`)

- 함수 레이블

- SP 조정(프레임 할당)

- 프레임 포인터 설정

- 함수 종료에서 SP 원복

- return (예: `jump RA`)

- 함수 끝 `pseudo` 명령어

```
// 어셈블리어 코드 생성 전 scaffold
return new Frame.Proc(
 "PROCEDURE " + name + "\n",
 body,
 "END " + name + "\n");
```

## 7.3 선언: Fragments (구현 예시: ProcedureIR)

변환 단계의 최종 산출물은 하나의 **Tree**가 아니라 여러 **fragment** 리스트, 각 **fragment**는 나중에 **assembly**로 번역될 독립 단위

함수 => ProcFrag(body, frame)

- **frame**: 지역 변수와 매개변수, 호출 규약 등 **machine-specific** 정보
- **body**: 함수 본문 Tree IR (procEntryExit1 거친 결과)

데이터 => DataFrag(data)

- 문자열 상수 등 데이터 영역에 들어갈 정보

## 7.3 선언: Fragments

두 가지 종류

- 프로시저 **Fragment**: `ProcFrag(Tree.Stm body, Frame.Frame frame)`
- 데이터 **Fragment**: `DataFrag(String data)`

```
package Translate;

public abstract class Frag {
 public Frag next; // fragment들을 연결 리스트로 관리
}

public ProcFrag(Tree.Stm body, Frame.Frame frame);
public DataFrag(String data);
```

## 7.3 선언: Fragments

### Translate 내부 구조

- `procEntryExit`: 함수 번역이 끝날 때 호출. `ProcFrag`를 생성하고 `fragment list`에 저장

- `getResult`: 누적된 `fragment list`를 반환

```
class Translate {
 private Frag frags; // accumulated fragment list

 public void procEntryExit(Exp body);
 public Frag getResult();
}
```

```
function header
↓
new Translate.Level(...)
↓
translate function body
↓
procEntryExit(body)
↓
ProcFrag(body, frame) 저장
↓
getResult()
↓
assembly generation
```

## 7.3 선언: 클래스와 객체

### MiniJava에서 객체 생성

- 멤버 변수들의 메모리 블록
- `new Vehicle()`은 레코드 생성과 유사
- 객체는 힙에 할당

`new Vehicle()`

-> 힙 공간 할당

-> 각 필드들을 0으로 초기화

메모리 구조:

offset 0: position

offset 1: gas

필요한 공간:

총 8 bytes

(2개 정수 x 4 bytes)

```
class Vehicle {
 int position;
 int gas;
 int move (int x) {
 position = position + x;
 return position;
 }
 int fill (int y) {
 gas = gas + y;
 return gas;
 }
}
```

---

**PROGRAM 7.5.** A MiniJava program.

---

## 7.3 선언: 클래스와 객체

MiniJava에서 `v.move()`

- `Vehicle_move(v)`로 해석
- `v`가 `this` 포인터로 전달
- 메서드는 `this`를 통해 해당 객체의 필드에 접근

메서드 이름

- `ClassName_methodName` (예: `Vehicle.move` -> `Vehicle_move`)

객체 변수 접근

- `field access = MEM(this + field_offset)`

|                       |                               |
|-----------------------|-------------------------------|
| <code>position</code> | -> <code>MEM(this + 0)</code> |
| <code>gas</code>      | -> <code>MEM(this + 4)</code> |

# [실습]

목표:

- class IRTranslator extends Visitor
- List<IRTranslator.ProcedureIR> translate(syntaxtree.Program root)

참고

- 트랜슬레이트 모듈 명세 (README\_Translate\_명세.md)
- chap7/Tree/{\*, Print.java} : IR 트리 언어에 대한 클래스 및 출력 함수
- IR 트리 변환을 단순하게 구현:: 교재에서 설명한 **Ex**, **Nx**, **Cx** 생성자 없이 진행

## IRTranslator 클래스 선언과 멤버 변수

```
28 public class IRTranslator implements Visitor {
29 private final Deque<HashMap<String, Temp>> envStack = new ArrayDeque<>();
30 private final Deque<HashMap<String, String>> typeEnvStack = new ArrayDeque<>();
31 private Stm resultStm; // last statement result
32 private Exp resultExp; // last expression result
33 private final ArrayList<ProcedureIR> procedures = new ArrayList<>();
34 private final HashMap<String, ArrayList<String>> classFields = new HashMap<>();
35 private final HashMap<String, HashMap<String, Integer>> fieldOffsets = new HashMap<>();
36 private final HashMap<String, HashMap<String, String>> classOwnFieldTypes = new HashMap<>();
37 private final HashMap<String, HashMap<String, String>> classAllFieldTypes = new HashMap<>();
38 private final HashMap<String, String> classParents = new HashMap<>();
39 private final HashMap<String, ArrayList<String>> classOwnFields = new HashMap<>();
40 private static final int WORD_SIZE = 4;
41 private String currentClassName = null;
```

## [실습] 컴파일러 메인 함수 Main.main()

```
21 public class Main {
22 public static void main(String [] args) {
23 boolean runOnly = false;
24 java.util.ArrayList<String> inputPaths = new java.util.ArrayList<>();
25 for (String arg : args) {
26 if ("--run-only".equals(arg)) runOnly = true;
27 else inputPaths.add(arg);
28 }
29
30 for (String path : inputPaths) {
31 try {
32 java.io.InputStream in = new java.io.FileInputStream(path);
33 MiniJavaParser parser = new MiniJavaParser(in);
34 Program root = parser.Goal();
35 if (!runOnly) root.accept(new PrettyPrintVisitor());
36 } catch (Exception e) {
37 e.printStackTrace();
38 }
39 }
40 }
41 }
```

## [실습] 컴파일러 메인 함수 Main.main()

```
30 for (String path : inputPaths) {
31 try {
32 java.io.InputStream in = new java.io.FileInputStream(path);
33 MiniJavaParser parser = new MiniJavaParser(in);
34 Program root = parser.Goal();
35 if (!runOnly) root.accept(new PrettyPrintVisitor());
36 TypeCheckVisitor tc = new TypeCheckVisitor();
37 boolean success = tc.check(root);
38 if (!success) {
39 System.err.println("Type check failed for " + path);
40 System.exit(-1);
41 }
42
43 // Translate to procedure-wise IR
44 IRTranslator.DEBUG = !runOnly;
45 IRTranslator tr = new IRTranslator();
46 List<IRTranslator.ProcedureIR> procedures = tr.translate(root);
```

## [실습] 컴파일러 메인 함수 Main.main()

```
45 IRTranslator tr = new IRTranslator();
46 List<IRTranslator.ProcedureIR> procedures = tr.translate(root);
47
48 Print printer = new Print(System.out);
49 Temp.TempMap tmap = new Temp.CombineMap(Frame.regNameMap(), new Temp.DefaultMap());
50
51 if (runOnly) {
52 System.out.println(".text");
53 System.out.println(".globl main");
54 }
55
56 for (IRTranslator.ProcedureIR p : procedures) {
57 if (!runOnly) System.out.println("Function " + p.name);
58
59 if (!runOnly) {
60 System.out.println("=== IR (unscheduled) ===");
61 printer.prStm(p.body);
62 }
```

## translate() 함수와 Fragment/ProcedureIR

```
47 public static class ProcedureIR {
48 public final String name;
49 public final Frame frame;
50 public final Stm body;
51
52 public ProcedureIR(String name, Frame frame, Stm body) {
53 this.name = name;
54 this.frame = frame;
55 this.body = body;
56 }
57 }
58
59 /** Translate entire program to procedure-wise IR units. */
60 public List<ProcedureIR> translate(syntaxtree.Program root) {
61 procedures.clear();
62 envPush();
63 root.accept(this);
64 envPop();
65 return new ArrayList<>(procedures);
66 }
```

## [실습] 프로그램

```
179 // Program
180 public void visit(syntaxtree.Program n) {
181 preScanClassLayouts(n);
182 n.m.accept(this);
183 for (int i = 0; i < n.cl.size(); i++) {
184 n.cl.elementAt(i).accept(this);
185 }
186 }
```

**Program**(MainClass m, ClassDeclList cl)

**MainClass**(Identifier i1, Identifier i2, Statement s)

abstract class ClassDecl

**ClassDeclSimple**(Identifier i, VarDeclList vl, MethodDeclList ml)

**ClassDeclExtends**(Identifier i, Identifier j,

VarDeclList vl, MethodDeclList ml) *see Ch.14*

classFields  
fieldOffsets  
classOwnFieldTypes  
classAllFieldTypes  
classParents  
classOwnFields

## [실습] 메인 클래스

```
257 // MainClass: translate single statement in main
258 public void visit(syntaxtree.MainClass n) {
259 envPush();
260 List<Stm> stms = new ArrayList<>();
261 n.s.accept(this);
262 if (resultStm != null) stms.add(resultStm);
263 Frame frame = new Frame("main", null);
264 resultStm = frame.procEntryExit1(maybeSeq(stms));
265 procedures.add(new ProcedureIR("main", frame, resultStm));
266 envPop();
267 }
```

**Program**(MainClass m, ClassDeclList cl)

**MainClass**(Identifier i1, Identifier i2, Statement s)

abstract class ClassDecl

**ClassDeclSimple**(Identifier i, VarDeclList vl, MethodDeclList ml)

**ClassDeclExtends**(Identifier i, Identifier j,

VarDeclList vl, MethodDeclList ml) *see Ch.14*

```

abstract class ClassDecl
 ClassDeclSimple(Identifier i, VarDeclList vl, MethodDeclList ml)
 ClassDeclExtends(Identifier i, Identifier j,
 VarDeclList vl, MethodDeclList ml) see Ch.14

```

## [실습] 클래스선언

```

269 // ClassDeclSimple: translate methods
270 public void visit(syntaxtree.ClassDeclSimple n) {
271 String savedClass = currentClassName;
272 currentClassName = n.i.s;
273 ArrayList<String> fields = new ArrayList<>();
274 for (int i = 0; i < n.vl.size(); i++) {
275 fields.add(n.vl.elementAt(i).i.s);
276 }
277 classFields.put(n.i.s, fields);
278 HashMap<String, Integer> offsets = new HashMap<>();
279 for (int i = 0; i < fields.size(); i++) {
280 offsets.put(fields.get(i), i * WORD_SIZE);
281 }
282 fieldOffsets.put(n.i.s, offsets);
283 for (int i = 0; i < n.ml.size(); i++) {
284 n.ml.elementAt(i).accept(this);
285 }
286 currentClassName = savedClass;
287 }

```

```

abstract class ClassDecl
 ClassDeclSimple(Identifier i, VarDeclList vl, MethodDeclList ml)
 ClassDeclExtends(Identifier i, Identifier j,
 VarDeclList vl, MethodDeclList ml) see Ch.14

```

## [실습] 클래스선언

```

289 // ClassDeclExtends: translate methods similarly
290 public void visit(syntaxtree.ClassDeclExtends n) {
291 String savedClass = currentClassName;
292 currentClassName = n.i.s;
293 ArrayList<String> fields = new ArrayList<>();
294 for (int i = 0; i < n.vl.size(); i++) {
295 fields.add(n.vl.elementAt(i).i.s);
296 }
297 // Inherit parent fields first if present
298 ArrayList<String> parentFields = classFields.getDefault(n.j.s, new ArrayList<>());
299 ArrayList<String> allFields = new ArrayList<>(parentFields);
300 allFields.addAll(fields);
301 classFields.put(n.i.s, allFields);
302 HashMap<String, Integer> offsets = new HashMap<>();
303 for (int i = 0; i < allFields.size(); i++) {
304 offsets.put(allFields.get(i), i * WORD_SIZE);
305 }
306 fieldOffsets.put(n.i.s, offsets);
307 for (int i = 0; i < n.ml.size(); i++) {
308 n.ml.elementAt(i).accept(this);
309 }
310 currentClassName = savedClass;
311 }

```

```

 VarDecl(Type t, Identifier i)
 MethodDecl(Type t, Identifier i, FormalList fl, VarDeclList vl,
 StatementList sl, Exp e)
 Formal(Type t, Identifier i)

```

## [실습] MethodDecl

```

315 // MethodDecl: body statements + return move
316 public void visit(syntaxtree.MethodDecl n) {
317 envPush();
318
319 // 목시적 인자 this와 메소드 인자의 갯수만큼 true인 boolean list
320 BoolList escapes = null;
321 for (int j = n.fl.size(); j >= 0; j--) {
322 escapes = new BoolList(false, escapes);
323 }
324
325 String mlabel = methodLabel(currentClassName, n.i.s);
326 Frame frame = new Frame(mlabel, escapes);
327
328 List<Stm> stms = new ArrayList<>();
329
330 List<Access> formals = frame.getFormals();
331
332 int formalIndex = 0;
333
334 // implicit this available inside methods
335 Access thisAcc = formals.get(formalIndex++);

```

```

 VarDecl(Type t, Identifier i)
 MethodDecl(Type t, Identifier i, FormalList fl, VarDeclList vl,
 StatementList sl, Exp e)
 Formal(Type t, Identifier i)

```

## [실습] MethodDecl

```

337 if (thisAcc instanceof InFrame) {
338 env().put("this", new Temp());
339 }
340 else if (thisAcc instanceof InReg) {
341 env().put("this", ((InReg)thisAcc).temp);
342 }
343 else {
344 throw new RuntimeException("Unexpected Access type for 'this'");
345 }
346 typeEnv().put("this", currentClassName);
347
348 // allocate temps for parameters and locals
349 for (int j = 0; j < n.fl.size(); j++) {
350 String name = n.fl.elementAt(j).i.s;
351 Access acc = formals.get(formalIndex++);
352 if (acc instanceof InFrame) {
353 env().put(name, new Temp());
354 } else if (acc instanceof InReg) {
355 env().put(name, ((InReg) acc).temp);
356 } else {
357 throw new RuntimeException("Unexpected Access type for parameter: " + name);
358 }
359 typeEnv().put(name, typeNameOf(n.fl.elementAt(j).t));
360 }

```

```

 VarDecl(Type t, Identifier i)
 MethodDecl(Type t, Identifier i, FormalList fl, VarDeclList vl,
 StatementList sl, Exp e)
 Formal(Type t, Identifier i)

```

## [실습] MethodDecl

```

362 for (int j = 0; j < n.vl.size(); j++) {
363 syntaxtree.VarDecl vd = n.vl.elementAt(j);
364 tempOf(vd.i.s);
365 typeEnv().put(vd.i.s, typeNameOf(vd.t));
366 }
367 // statements
368 for (int i = 0; i < n.sl.size(); i++) {
369 n.sl.elementAt(i).accept(this);
370 if (resultStm != null) stms.add(resultStm);
371 }
372 // return value
373 n.e.accept(this); Exp ret = resultExp;
374 stms.add(new MOVE(new TEMP(Frame.RV), ret));
375
376 resultStm = frame.procEntryExit1(maybeSeq(stms));
377 procedures.add(new ProcedureIR(mlabel, frame, resultStm));
378
379 envPop();

```

VarDecl(Type t, Identifier i)

MethodDecl(Type t, Identifier i, FormalList fl, VarDeclList vl,  
StatementList sl, Exp e)

Formal(Type t, Identifier i)

## [실습] MethodDecl

```
381 if (DEBUG) {
382 System.out.println("Method: " + mlabel);
383 System.out.println("Formals:");
384 for (int i = 0; i < formals.size(); i++) {
385 String name;
386 if (i == 0) {
387 name = "this";
388 } else {
389 name = n.fl.elementAt(i - 1).i.s;
390 }
391
392 Access acc = formals.get(i);
393 if (acc instanceof InFrame) {
394 System.out.println(name + "\t" + i + ": InFrame(offset=" + ((InFrame) acc).offset + ")");
395 } else if (acc instanceof InReg) {
396 System.out.println(name + "\t" + i + ": InReg(temp=" + ((InReg) acc).temp + ")");
397 } else {
398 System.out.println(name + "\t" + i + ": Unknown Access type");
399 }
400 }
401 System.out.println("tempMapString:");
402 System.out.println(frame.tempMapString()); // Print temp mapping for debugging
403 }
404 }
```

## [실습] visit(문장)

```
452 public void visit(syntaxtree.Print n) {
453 n.e.accept(this); Exp e = resultExp;
454 resultStm = print1(e);
455 }
456
457 public void visit(syntaxtree.Assign n) {
458 n.e.accept(this); Exp rhs = resultExp;
459 resultStm = assignVar(n.i.s, rhs);
460 }
461
462 public void visit(syntaxtree.ArrayAssign n) {
463 // a[i] = v ==> MOVE(MEM(a + i*4), v)
464 n.e1.accept(this); Exp idx = resultExp;
465 n.e2.accept(this); Exp val = resultExp;
466 Exp base = varExp(n.i.s);
467 Exp addr = bin(BINOP.PLUS,
468 base,
469 bin(BINOP.PLUS, new CONST(4), bin(BINOP.MUL, idx, new CONST(4))));
470 resultStm = new MOVE(new MEM(addr), val);
471 }
```

## [실습] visit(문장)

```
423 public void visit(syntaxtree.If n) {
424 n.e.accept(this); Exp cond = resultExp;
425 Label lt = new Label(); Label lf = new Label(); Label le = new Label();
426 // if (cond != 0) then s1 else s2
427 CJUMP c = new CJUMP(CJUMP.NE, cond, const0(), lt, lf);
428 n.s1.accept(this); Stm s1 = resultStm;
429 n.s2.accept(this); Stm s2 = resultStm;
430 resultStm = seq(
431 c,
432 new LABEL(lt), s1,
433 new JUMP(le),
434 new LABEL(lf), s2,
435 new LABEL(le)
436);
437 }
```

## [실습] visit(식)

```
553 public void visit(syntaxtree.IntegerLiteral n) { resultExp = new CONST(n.i); }
554 public void visit(syntaxtree.True n) { resultExp = const1(); }
555 public void visit(syntaxtree.False n) { resultExp = const0(); }
556
557 public void visit(syntaxtree.IdentifierExp n) { resultExp = varExp(n.s); }
558
559 public void visit(syntaxtree.This n) { resultExp = new TEMP(tempOf("this")); }
```

## [실습] visit(식)

```
473 // Expressions
474 public void visit(syntaxtree.And n) {
475 n.e1.accept(this); Exp a = resultExp;
476 n.e2.accept(this); Exp b = resultExp;
477 resultExp = bin(BINOP.AND, a, b);
478 }
479
480 public void visit(syntaxtree.LessThan n) {
481 // Lower to ESEQ that sets a temp to 1 if e1<e2 else 0
482 n.e1.accept(this); Exp a = resultExp;
483 n.e2.accept(this); Exp b = resultExp;
484 Temp t = new Temp(); Label lt = new Label(); Label lf = new Label(); Label le = new Label();
485 Stm s = seq(
486 new MOVE(new TEMP(t), const0()),
487 new CJUMP(CJUMP.LT, a, b, lt, lf),
488 new LABEL(lt), new MOVE(new TEMP(t), const1()),
489 new JUMP(le),
490 new LABEL(lf),
491 new LABEL(le)
492);
493 resultExp = new ESEQ(s, new TEMP(t));
494 }
```

## [실습] visit(식)

```
514 public void visit(syntaxtree.ArrayLookup n) {
515 n.e1.accept(this); Exp base = resultExp;
516 n.e2.accept(this); Exp idx = resultExp;
517 Exp addr = bin(BINOP.PLUS,
518 base,
519 bin(BINOP.PLUS, new CONST(4), bin(BINOP.MUL, idx, new CONST(4))));
520 resultExp = new MEM(addr);
521 }
522
523 public void visit(syntaxtree.ArrayLength n) {
524 n.e.accept(this); Exp arr = resultExp;
525 resultExp = new MEM(arr);
526 }
```

## [실습]

```
528 public void visit(syntaxtree.Call n) {
529 // Lower to CALL(label, receiver, args...)
530 String recvClass = receiverClassOf(n.e);
531 n.e.accept(this); Exp recv = resultExp;
532 ExpList args = new ExpList(recv, null);
533 for (int i = 0; i < n.el.size(); i++) {
534 n.el.elementAt(i).accept(this);
535 // append to args list tail
536 args = append(args, resultExp);
537 }
538 if (recvClass == null)
539 System.err.println("Warning: unable to resolve receiver class for method call."
540 + "Treating method name as global function.");
541 String callName = (recvClass != null) ? methodLabel(recvClass, n.i.s) : n.i.s;
542 resultExp = new CALL(name(callName), args);
543 }
```

## [실습]

```
561 public void visit(syntaxtree.NewArray n) {
562 // Runtime alloc: AllocArray(length) stores length at offset 0 and
563 // initializes all elements [1..length] to 0.
564 n.e.accept(this); Exp size = resultExp;
565 Temp t = new Temp();
566 Stm s = new MOVE(new TEMP(t), new CALL(name(Frame.allocArrayLabel()), new ExpList(size, null)));
567 resultExp = new ESEQ(s, new TEMP(t));
568 }
569
570 public void visit(syntaxtree.NewObject n) {
571 String cname = n.i.s;
572 int fcount = classFields.getDefault(cname, new ArrayList<>()).size();
573 int sizeBytes = fcount * WORD_SIZE;
574 Temp t = new Temp();
575 List<Stm> inits = new ArrayList<>();
576 // allocate object space (sizeBytes)
577 inits.add(new MOVE(new TEMP(t), new CALL(name(Frame.heapAllocLabel()), new ExpList(new CONST(sizeBytes), null))));
578 // zero-initialize fields
579 for (int i = 0; i < fcount; i++) {
580 int off = i * WORD_SIZE;
581 inits.add(new MOVE(new MEM(bin(BINOP.PLUS, new TEMP(t), new CONST(off))), const0()));
582 }
583 resultExp = new ESEQ(seq(inits), new TEMP(t));
584 }
```

## [실습]

함수 `procEntryExit1`(함수 몸체를 변환하여 얻은 IR 트리 `Tree.Stm`)

- view shift 코드
- callee-save 레지스터 저장: `MOVE(TEMP(saved), TEMP(reg))`
- 함수 몸체를 변환하여 얻은 IR 트리 `Tree.Stm`
- callee-save 레지스터 복구 (저장 순서의 역순): `MOVE(TEMP(reg), TEMP(saved))`